

6.104 · Software Design · MIT · Fall 2026

design case studies

Daniel Jackson

what you'll learn today

concept design in action: 3 real problems
a common theme: disentangling

*restaurant
reservations*

a basic reserving concept

concept Reserving [User, Resource]

purpose secure future access to a scarce resource

principle A user can reserve a resource and then claim it later

state

a set of Reservations with

a User

a Resource

a Claimed set of Reservations

actions

reserve (user: User, resource: Resource):

returns (reservation: Reservation)

cancel (reservation: Reservation)

claim (reservation: Reservation)

what bad situation
does this concept
prevent?

what kinds of
resources would this
work for?

what is the spec for
the claim action?

which actor typically
executes each of the
actions?

a basic reserving concept with no-shows

concept Reserving [User, Resource]

purpose secure future access to a scarce resource

principle A user can reserve a resource and then claim it later, or be a no-show

state

a set of Reservations with

a User

a Resource

a Claimed set of Reservations

a NoShows set of Reservations

actions

reserve (user: User, resource: Resource):

returns (reservation: Reservation)

cancel (reservation: Reservation)

claim (reservation: Reservation)

noShow (reservation: Reservation)

discussion:
why noShow?

what does OpenTable
do if you noShow?

a provisioning concept

concept Provisioning [Provider]

purpose provision slots for booking in advance

principle slots can be made and then booked

state

a set of Slots with
a Provider
a start DateTime
an end DateTime
a Booked set of Slots

actions

makeSlot (provider: Provider, start: DateTime, end: DateTime):
returns (slot: Slot)
where end is after start
then create new slot
removeSlot (slot: Slot)
where slot is in slots
then remove slot
book (slot: Slot)
where slot is not in Booked
then add slot to Booked

what's tricky
about the action
removeSlot?

how a reservation is made

when `Requesting.request (reserve, provider, user, start)`

where

Provisioning: slot not in Booked for provider and start

then

`Reserving.reserve (user, slot): (reservation: Reservation)`

`Requesting.respond (reservation)`

what's missing for
practical restaurant
reservations?

concept Reserving [User, Resource]

actions

`reserve (user: User, resource: Resource):`

`returns (reservation: Reservation)`

`cancel (reservation: Reservation)`

`claim (reservation: Reservation)`

`noShow (reservation: Reservation)`

concept Provisioning [Provider]

state

a set of Slots with

a Provider

a start DateTime

an end DateTime

a Booked set of Slots

adding party sizes

concept Reserving [User, Resource]

purpose secure future access to a scarce resource

principle A user can reserve a resource and then claim it later.

state

a set of Reservations with

a User

a Resource

a party Number

a Claimed set of Reservations

actions

reserve (user: User, resource: Resource, **party: Number**):

returns (reservation: Reservation)

where no reservation for resource

then create new reservation for user, resource **and party**

...

concept Provisioning [Restaurant]

purpose provision slots for booking in advance

principle slots can be made and then booked

state

a set of Slots with

a Provider

a start DateTime

an end DateTime

a minParty Number

a maxParty Number

actions

makeSlot (provider: Provider, start:

DateTime, end: DateTime, **minParty:**

Number, maxParty: Number): (slot: Slot)

where end is after start, **max >= min**

then create new slot with ...

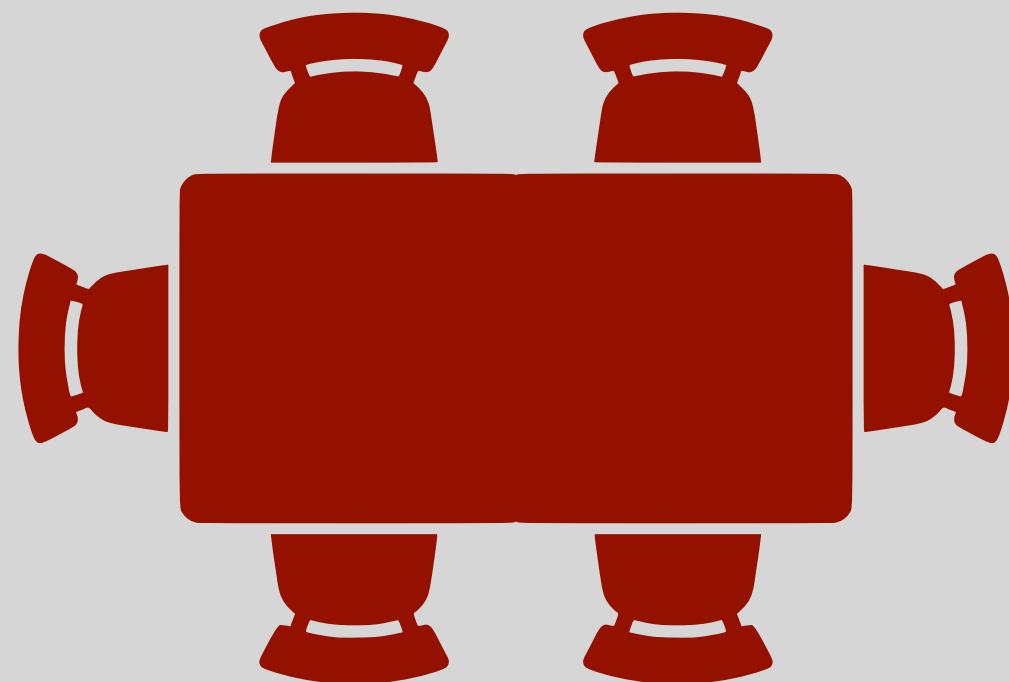
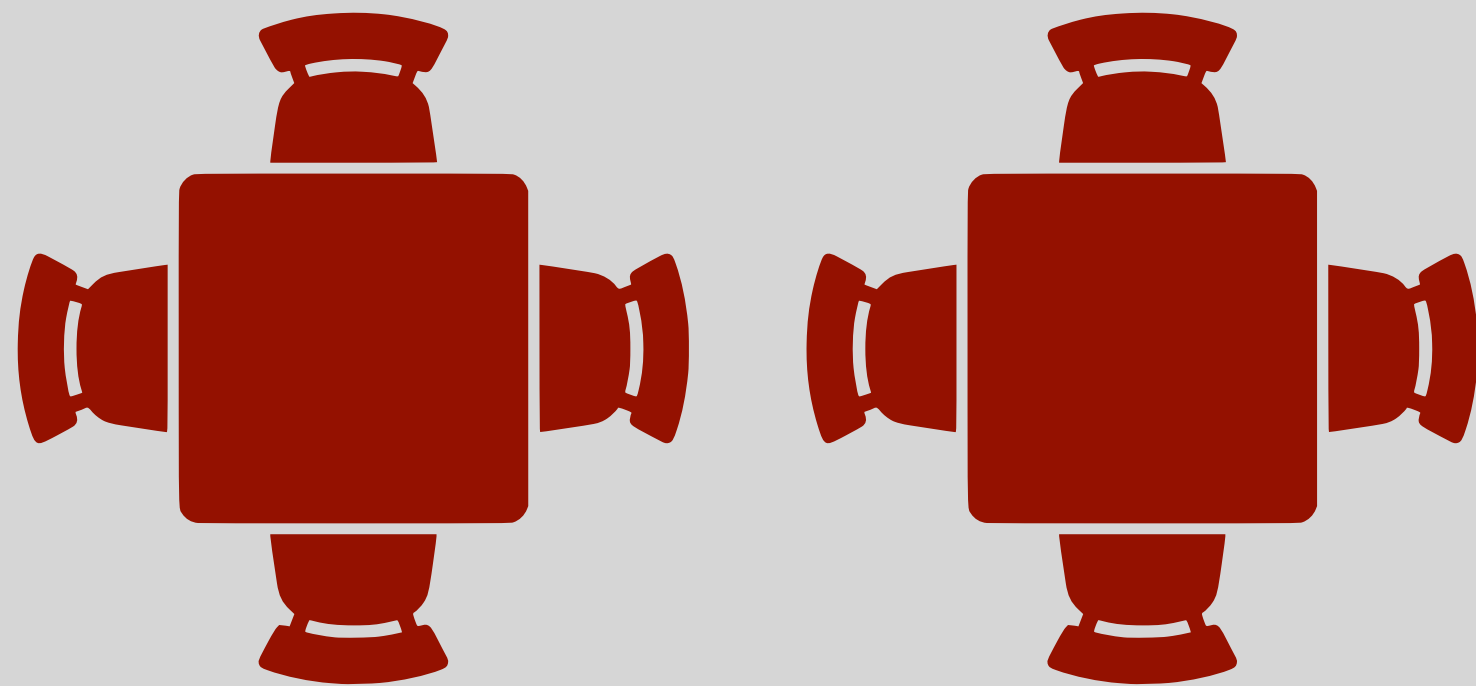
...

greedy slot selection

```
when Requesting.request (reserve, provider, user, start)
where
  Provisioning: slot not in Booked for provider and start
    and minParty <= party <= max
    and no other slot with smaller maxParty
then
  Reserving.reserve (user, slot, party): (reservation: Reservation)
  Requesting.respond (reservation)
```

reconfiguring slots

can you modify the state and actions of the Provisioning concept to allow this?



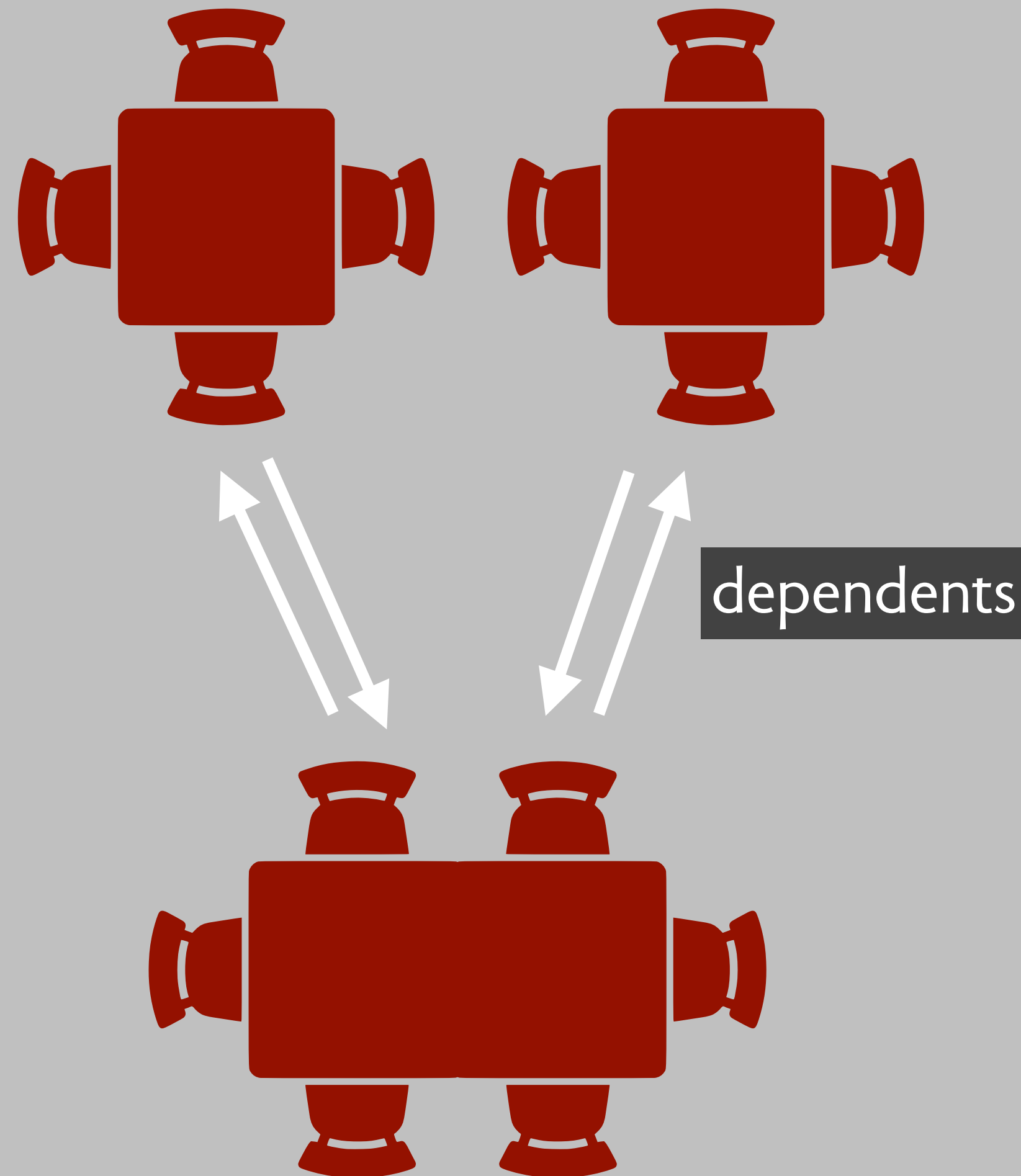
state

a set of Slots with
a Provider
a start DateTime
an end DateTime
a Booked set of Slots

actions

makeSlot (provider: Provider, start: DateTime, end: DateTime, minParty: Number, maxParty: Number): (slot: Slot)
where end is after start, max $\geq$ min
then create new slot with ...

...



concept Provisioning [Restaurant]

purpose provision slots for booking in advance

principle slots can be made and then booked

state

a set of Slots with

a dependents set of Slots

a Provider

a start DateTime

an end DateTime

a minParty Number

a maxParty Number

actions

addDependent (slot: Slot, dependent: Slot)

book (slot: Slot)

where slot is not in Booked

then add slot **and its dependents** to Booked

what did this separation buy us?

state

a set of Reservations with
a User
a Resource
a Claimed set of Reservations

concept Reserving [User, Resource]

purpose secure future access to a scarce resource

principle A user can reserve a resource and then claim it later

state

a set of Slots with
a Provider
a start DateTime
an end DateTime
a Booked set of Slots

concept Provisioning [Provider]

purpose provision slots for booking in advance

principle slots can be made and then booked

why not this?

state

a set of Slots with
a Provider
a start DateTime
an optional Reserver

concept Provisioning [Provider, Reserver]

simpler

understand &
implement concepts
independently

more reusable

reserving without
timed slots, eg

stakeholders

customer vs provider:
affects access & design

Google's booking API

Booking

A booking for an inventory slot

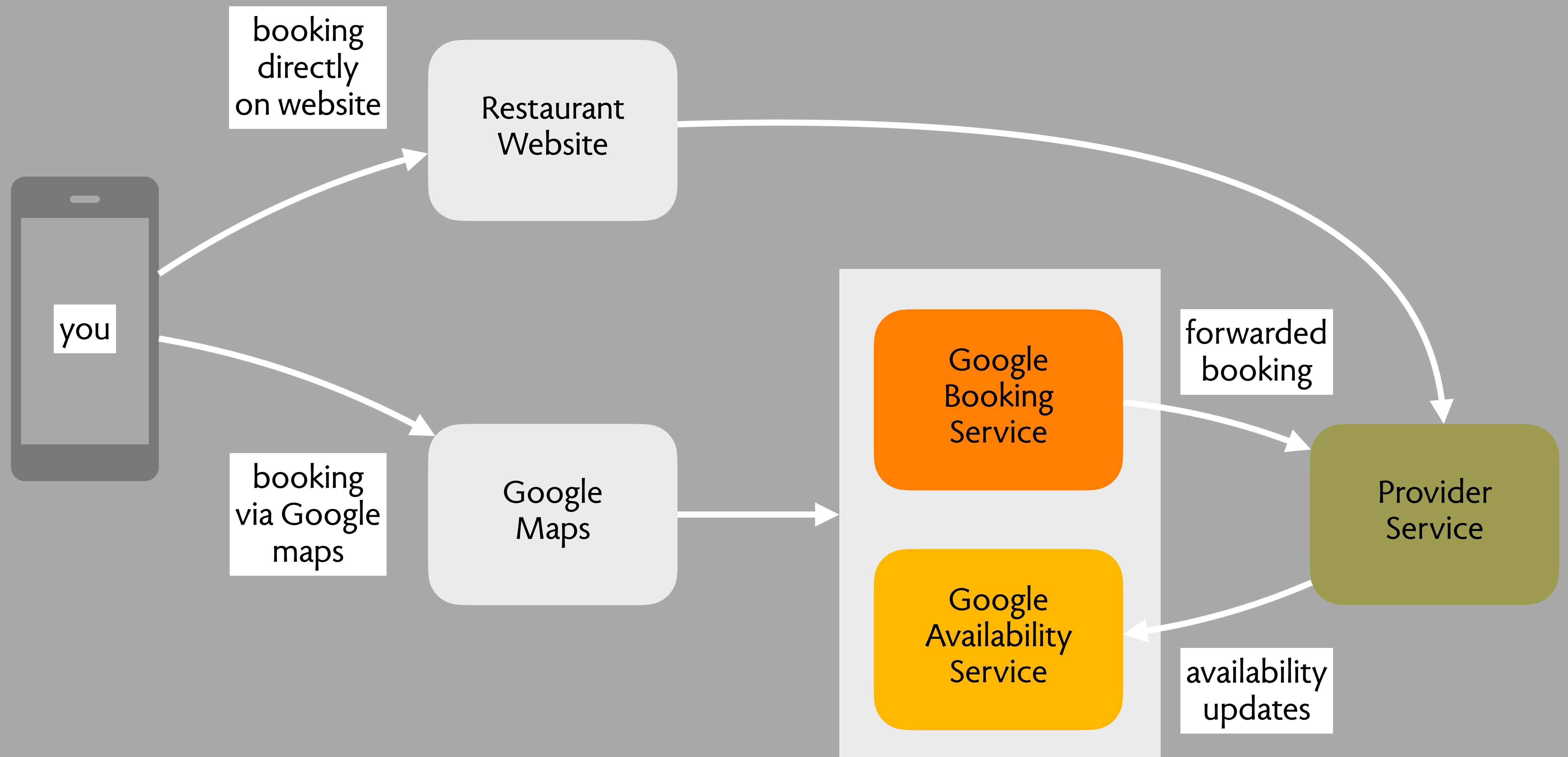
Fields	
name	string Resource name of the booking: <code>partners/{partner ID}/bookings/{booking ID}</code>
merchant_id	string ID of the merchant for the slot
service_id	string ID of the merchant service
start_time	Timestamp Start time of the appointment slot
duration	Duration Duration of the appointment slot
client_information	ClientInformation Personal information of the client making the appointment
status	BookingStatus Status of the booking
payment_information	PaymentInformation Information about payment transactions that relate to the booking.
party_size	int64 Party size of the booking

Availability

An availability slot of the merchant's service, indicating time and number of spots.

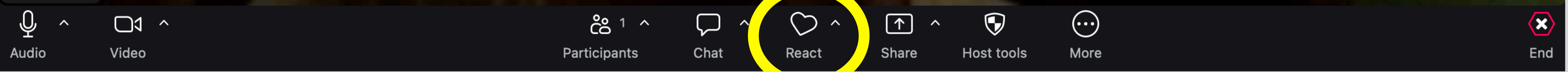
Fields	
start_time	Timestamp Start time of the appointment slot.
duration	Duration Duration of the appointment slot
spots_total	int64 Number of total spots and open spots of this availability. Examples: - Yoga class of 10 spots with 3 booked: <code>availability {spots_total: 10 ...}</code> - Chair massage session which is already fully booked: <code>availability {spots_total: 10 ...}</code> Note: If sending requests using the availability compression format defined below, - A Recurrence implies <code>spots_total=1</code> and <code>spots_open=1</code>. - A ScheduleException implies <code>spots_total=1</code> and <code>spots_open=0</code>.
spots_open	int64 Number of open spots.
availability_tag	string An optional opaque string to identify this availability slot. If set, it will be included in book/update/cancel appointments.
resources	Resources Optional resources used to disambiguate this availability slot from others when doing book/update/cancel appointments. E.g. the same Yoga class with two 2 instructors:

how Google's booking API works

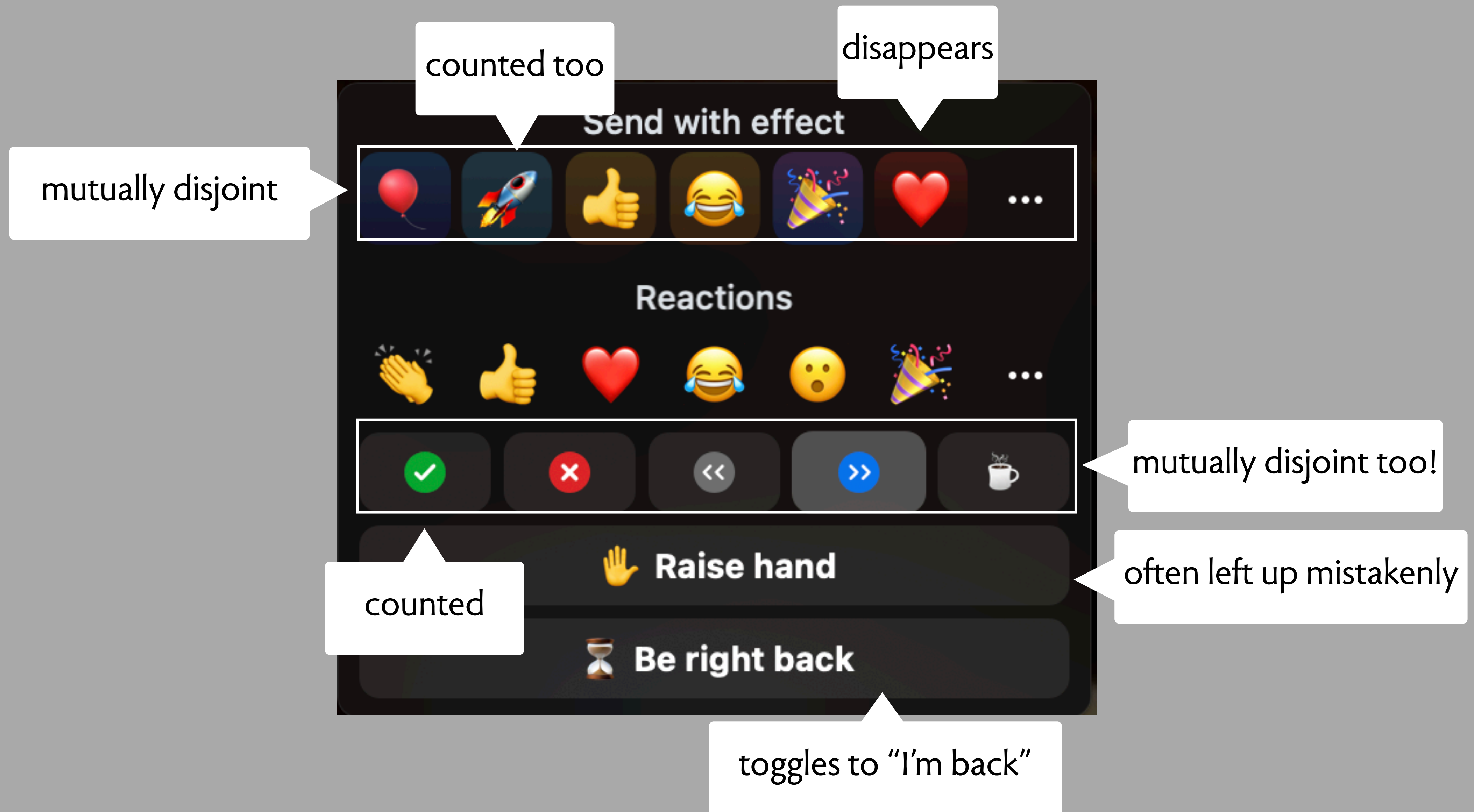


zoom's reactions

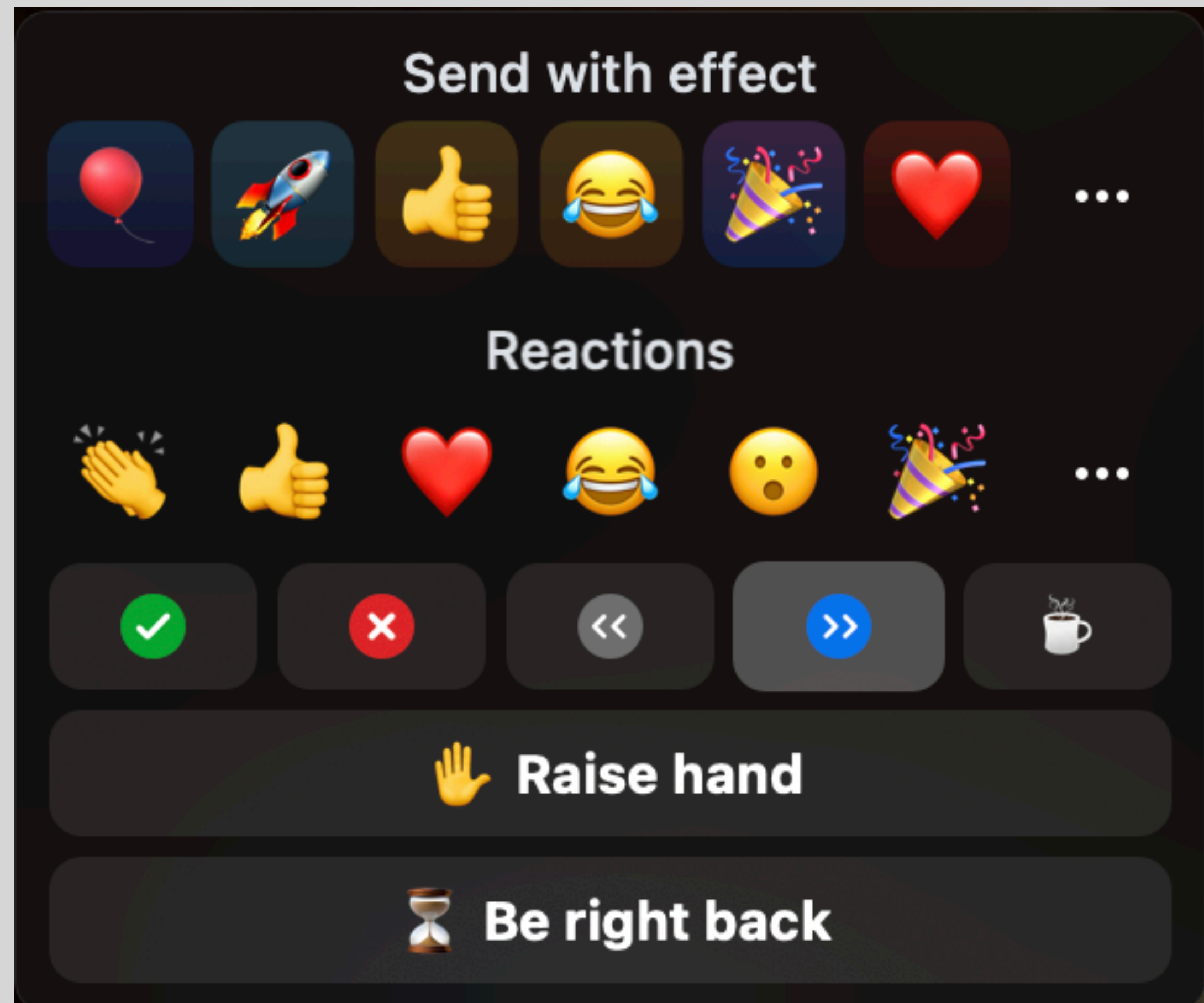
Zoom's reactions



anomalous behaviors



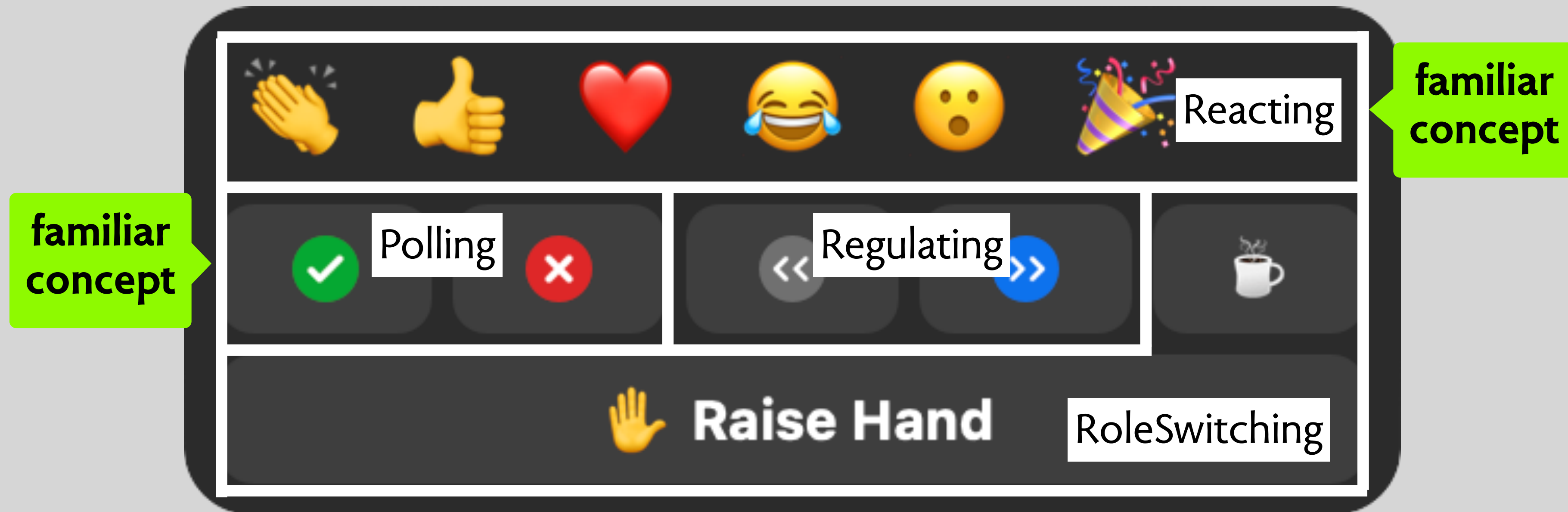
how would you fix this design?

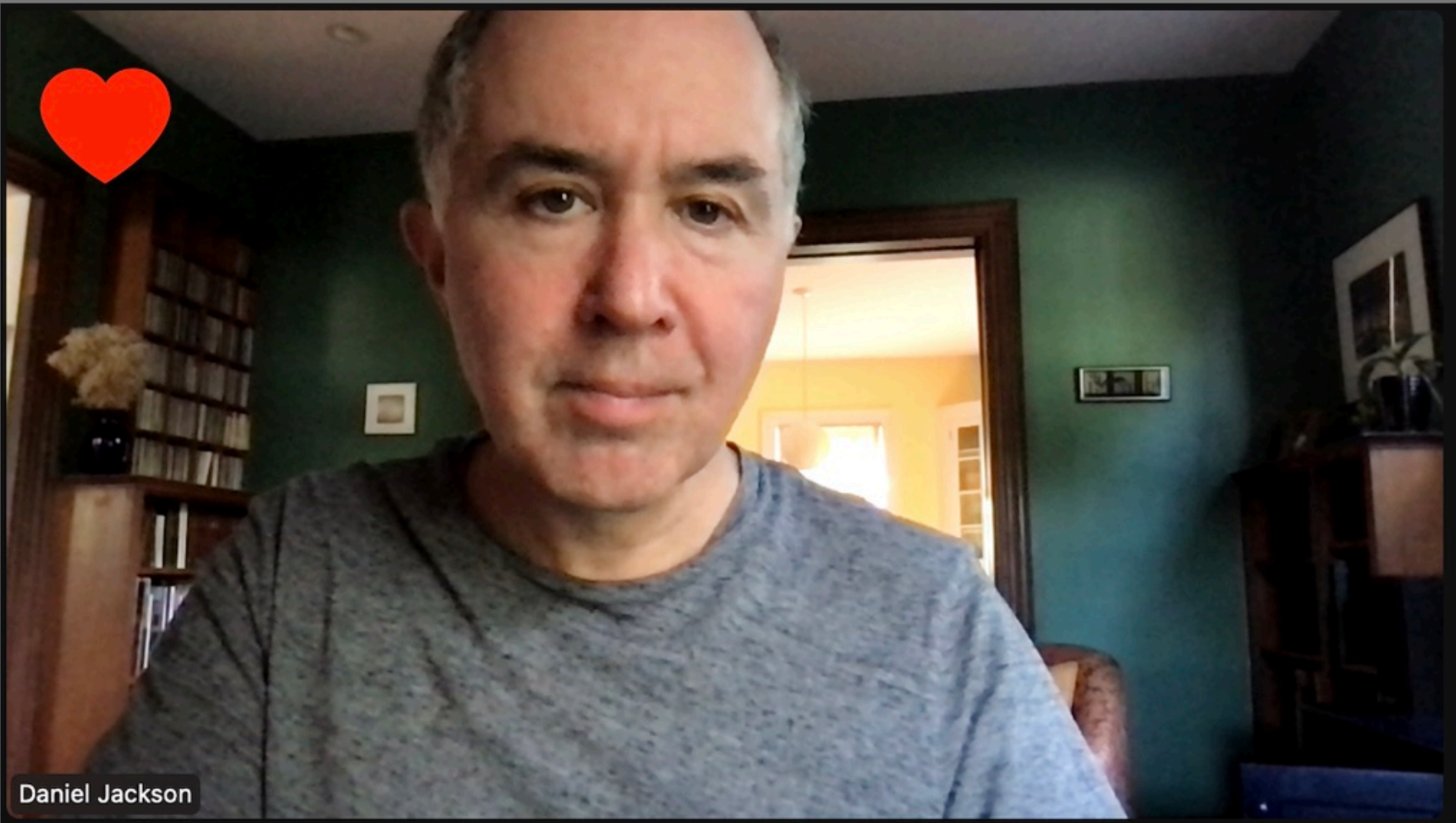


BY6JV3

<https://class.mit-sdg.dev/join>

my take: splitting into coherent concepts





RoleSwitching

Chatting

Reacting

Regulating

Request to speak

Watching/listening

Speaking

I'm away

Audience

Recent emoji

Other emoji

Everyone ▾

Type here...

...

Slow down

Just right

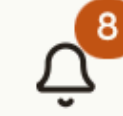
Speed up

16<<

24⦿

9>>

*private forum
discussions*



New discussion

Title

What would you like to discuss?

Opening post

B *I* ↺ <> ☞ ☰

⌘↵ to post

Write your question or idea...



Post discussion



bad situation: student wants to ask staff a question but doesn't want everyone to see it



New discussion

Who can see this?

Staff privately ▾



Visible to Staff

Title

What would you like to discuss?

Opening post

B *I* ↺ <> 📄 ☰

⌘↵ to post

Write your question or idea...



Post discussion



what's wrong with this design?

staff-private discussions: just add a flag to state in discussion concept?

how would you do it?

Authenticating

Let a person create an account with credentials, then use those credentials to identify themselves later and be found again by that address.

Rolling

Define roles as named sets of capabilities, then give each user at most one role within a context.

Rostering

Keep one revisable class configuration, together with sections and pending, active, or dropped seats held by its members ...

Conversing

Arrange items into conversations, with one root and replies that may have replies of their own.

Revising

Keep numbered versions of an item's content so readers can compare its current and earlier text.

Posting

Let an author create, edit, and delete a post while retaining its author and creation time.

Grouping

Keep a collection of people identified by an independent group identity and equal-power roster, so members can rename the group and add and remove members etc.

step 1

brainstorm ideas,
then ask questions
about the concepts

step 2

propose a design:
modify or add a
concept? a reaction?

a new concept

concept Accessing [Holder, Resource]

purpose establish the complete set of holders for a resource and keep that choice until the resource is retired.

principle Mira establishes access to a room for Omar and Lee together. She queries its holders and finds both. An attempt to establish the room again for Ravi is refused, leaving Omar and Lee in place. When the room closes, Mira retires its access. Its holders query then returns no row. Repeating retirement succeeds, but a delayed establishment for the same room is refused.

state

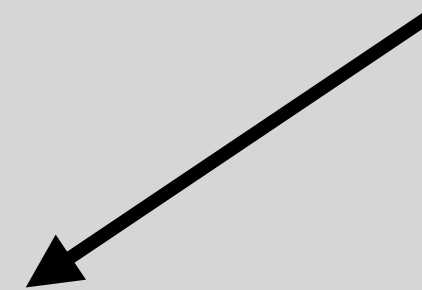
a set of Establishments with
a resource Resource
a holders set of Holder
a set of Retirements with
a resource Resource

actions

establish(resource: Resource, holders: set Holder) : return (resource: Resource)
where resource has neither an establishment nor a retirement
then add one establishment with resource and holders and return resource
_holders (resource: Resource) : optional (holders: set Holder)

what are the holders and resources?

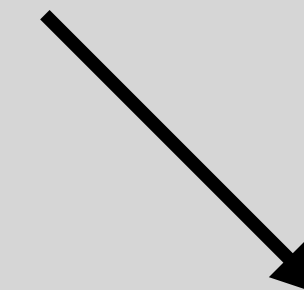
concept Accessing [Holder, Resource]



Authenticating.User

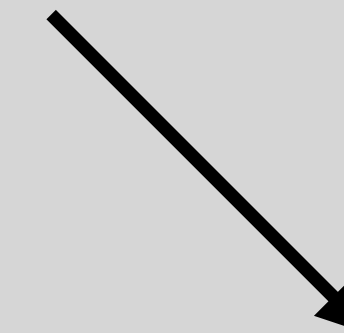
Grouping.Group

*a standing audience
eg: everyone includes
members in Rostering*



Conversing.Conversation

concept Conversing [Item]



Posting.Post

everyone
can "everyone"
be a group?

where we ended up

New discussion

Who can see this?

Everyone in the course ▾

🌐 Visible to everyone

Title

Team pitches

Opening post

B *I* ↺ ↻

Write your question

🌐 Everyone in the course

🔒 Staff privately

Choose people or groups...

Find people, groups, or sections...

👤 👤 👤 👤

👤 👤 👤 👤

👤 👤 👤 👤

6.1040 pre-semester prep Group

6.1040 Staff Group

Staff Course audience

Students Course audience

Done

⌘↵ to post

Post discussion



key ideas of
concept design



authentic demand
bad situations
not just features

customer journey
pain points



granular function
separate & reusable
purpose-driven

microservices
Amazon etc (2010s)



coordination
explicit causal links
no embedded calls

event-driven arch
very widely used



relational state
simpler than objects
view structuring

OOP successors
entity-component

parting
thought

looking for disentangling opportunities

disentangling gives simplicity and power

Zoom: making reactions distinct concepts

Forum: accessing and groups

Reserving: reserving vs. provisioning

in your personal projects

can you find ways to disentangle?